

CalmAn Detailed Modification

Overview

This modification introduces the `fit_v2` method, which re-optimizes temporal components C while **keeping spatial components A fixed**. This is particularly useful for scenarios where high-quality spatial components are already available (e.g., manually annotated ROIs).

Modified Files List

1. cnmf.py - Core Interface File

Modification 1.1: `fit_v2` Method

Functionality:

- Similar workflow to standard `fit` method, but uses fixed-A versions
- Calls `initialize_with_A_fixed` instead of `initialize` during initialization
- Uses specialized fixed-A functions for spatial/temporal updates

Key Code Segment:

```
def fit_v2(self, images, indices=(slice(None), slice(None))) -> None:
    # ... data preparation code same as fit ...

    if self.estimated.A is None:
        logger.info('initializing ...')
        self.initialize(Y)
    elif (not (self.estimated.A is None)):
        logger.info('initializing: A fixed mode...')
        self.initialize_with_A_fixed(Y) # <-- Key difference

    # ... subsequent update workflow ...
```

Main Differences from `fit`:

1. Check if A exists, if so call `initialize_with_A_fixed`
2. Rest of the workflow is similar but calls fixed-A versions of update functions

Modification 1.2: `initialize_with_A_fixed` Method

Functionality:

- Initialize using existing A

- Calls `initialize_components_fixed_A` function

Key Code:

```
def initialize_with_A_fixed(self, Y, **kwargs) -> None:
    """Component initialization with fixed A"""
    self.params.set('init', kwargs)
    estim = self.estimated

    if (self.params.get('init', 'method_init') == 'corr_pnr' and
        self.params.get('init', 'ring_size_factor') is not None):
        A_temp = self.estimated.A.copy() # Copy existing A
        estim.A, estim.C, estim.b, estim.f, estim.center, \
            extra_lp = initialize_components_fixed_A(
                Y, A=A_temp, sn=estim.sn,
                options_total=self.params.to_dict(),
                **self.params.get_group('init'))
        # ... handle extra outputs ...
```

2. initialization.py - Initialization Module

Modification 2.1: `initialize_components_fixed_A`

Functionality:

- Initialize other components (C, b, f, etc.) with A fixed
- Primarily for 1-photon data (corr_pnr method)

Signature:

```
def initialize_components_fixed_A(Y, A, K=30, gSig=[5, 5],
                                # ... other parameters ...
                                ):
    pass
```

Key Parameters:

- `Y`: Raw data
- `A`: Fixed spatial components (required)
- Other parameters same as standard `initialize_components`

Workflow:

```
if method == 'corr_pnr':
    Ain, Cin, _, b_in, f_in, extra_lp = greedyROI_corr_fixed_A(
        Y, Y_ds, A_fixed=A, # <-- Pass fixed A
        max_number=K, gSig=gSig[0], gSig=gSig[0],
```

```

        # ... other parameters ...
    )

```

Modification 2.2: `greedyROI_corr_fixed_A` Function

Functionality:

- Initialize using `corr_pnr` method with fixed A
- Calls `init_neurons_corr_pnr_fixed_A` for initialization
- Uses fixed-A versions of spatial and temporal update functions

Key Call Chain:

```

def greedyROI_corr_fixed_A(Y, Y_ds, A_fixed, ...):
    # 1. Initialize neurons with fixed A
    A, C, _, _, center = init_neurons_corr_pnr_fixed_A(
        Y_ds, A_fixed=A_fixed, gSiz=gSiz, gSig=gSig,
        # ...
    )

    # 2. Update spatial components (fixed-A version)
    A, _, C, _ =
    caiman.source_extraction.cnmf.spatial.update_spatial_components_fixed_A_ljr(
        B, C=C, f=..., A_in=A,
        # ...
    )

    # 3. Update temporal components (fixed-A version)
    C, A =
    caiman.source_extraction.cnmf.temporal.update_temporal_components_fixed_A(
        B, spr.csc_matrix(A, dtype=np.float32),
        # ...
    )

```

Multiple Iterations:

- First iteration: On downsampled data (`Y_ds`)
- Second iteration: On full resolution data (`Y`)
- Each iteration updates spatial then temporal components

3. `spatial.py` - Spatial Component Update Module

Modification: `update_spatial_components_fixed_A` Function

Functionality:

- Update spatial components with A fixed as constraint

Signature:

```
def update_spatial_components_fixed_A(Y, C=None, f=None, A_in=None,
                                       sn=None, dims=None, ...):
```

Key Features:

- Accepts `A_in` as the fixed spatial component input
- Does not change A's shape or sparse structure
- Mainly used for iterative optimization during initialization

4. temporal.py - Temporal Component Update Module

Modification: `update_temporal_components_fixed_A` Function (Lines 384-464)

Functionality:

- **Core Modification:** Fix A and b, only update temporal components C and f
- Removed "delete empty components" code block to ensure A shape remains unchanged

Workflow:

```
def update_temporal_components_fixed_A(Y, A, b, Cin, fin, ...):
    # 1. Data preparation
    A_tot = scipy.sparse.hstack((A, b)).tocsc()

    # 2. Calculate projections
    YA = A_tot.T.dot(Y).T * diags(1. / nA)
    AA = (A_tot.T.dot(A_tot)) * diags(1. / nA)
    YrA = YA - AA.T.dot(Cin_tot).T

    # 3. Iterative optimization (calls update_iteration)
    C_tot, S, bl, YrA, cl, sn, g, lam = update_iteration(
        parrllcomp, len_parrllcomp, nb, C_tot, S, bl, nr,
        ITER, YrA, cl, sn, g, Cin_tot, T, nA, dview, debug, AA, kwargs
    )

    # 4. [KEY] Don't delete empty components, directly split results
    C_out = C_tot[:nr, :]
    # ... return results ...
```

Core Design Philosophy

1. Fixed-A Constraint Propagation

```
fit_v2()  
  ↳ initialize_with_A_fixed()  
    ↳ initialize_components_fixed_A()  
      ↳ greedyROI_corr_fixed_A()  
        ↳ init_neurons_corr_pnr_fixed_A()  
        ↳ update_spatial_components_fixed_A()  
        ↳ update_temporal_components_fixed_A()
```

Throughout the entire call chain, A is always passed as a fixed parameter and never modified.

2. Key Constraint Points

1. **Initialization Phase:** Use existing A, don't regenerate
2. **Spatial Update:** Use special functions to keep A unchanged
3. **Temporal Update:** Remove delete-empty-components logic to ensure A's count and shape unchanged
4. **Deconvolution:** Performed under fixed-A constraint

3. Applicable Scenarios

- Already have high-quality spatial components A (e.g., manually annotated ROIs)
- Need to re-analyze using same ROIs across different time periods
- Want to test the impact of fixed spatial components on results
- 1-photon calcium imaging data (corr_pnr method)

Usage Notes

1. Prerequisites

- Must first run standard `fit()` once, or obtain A through other means
- `estimates.A` cannot be `None`
- Recommended to use `method_init='corr_pnr'`
- These modifications are based on CalmAn version 13.0.
- `fit_v2` is primarily an optimization for 1p data (corresponding to the case where nb=0 in `fit`).

2. Parameter Settings

```
params = CNMFParams(params_dict={
    'init': {
        'method_init': 'corr_pnr', # Required
        'ring_size_factor': 1.5,   # Must not be None
        # ... other parameters ...
    }
})
```

3. Workflow Example

```
# Step 1: Standard fit
cnm = CNMF(n_processes=1, params=params)
cnm.fit(images)
A_original = cnm.estimateds.A.copy()
C_original = cnm.estimateds.C.copy()

# Step 2: fit_v2 (using existing A)
cnm2 = CNMF(n_processes=1, params=params)
cnm2.estimateds.A = A_original # Set fixed A
cnm2.fit_v2(images)
C_new = cnm2.estimateds.C # New temporal traces

# Step 3: Compare
difference = np.abs(C_original - C_new)
```